Experiment 4: Autoencoder for Data Compression

➢ Objective

To design and implement an **autoencoder neural network** for compressing image data (MNIST digits) and reconstructing it back, understanding unsupervised feature learning.

➢ Prerequisites

- Basics of neural networks
- Concept of dimensionality reduction
- Familiarity with Keras/TensorFlow

➢ Theory Background

Autoencoders are an unsupervised learning technique in which we leverage neural networks for the task of representation learning. Specifically, we'll design a neural network architecture such that we impose a bottleneck in the network which forces a compressed knowledge representation of the original input.

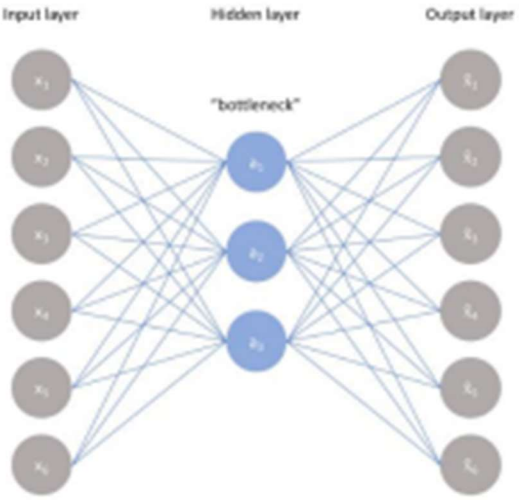


Image Credit: Abeyaantrix Edusoft

An **Autoencoder** is a type of neural network used for **unsupervised learning of efficient codings**.
It consists of two parts:

1. **Encoder**: Compresses the input into a smaller representation (latent space).
2. **Decoder**: Reconstructs the input from the compressed form.

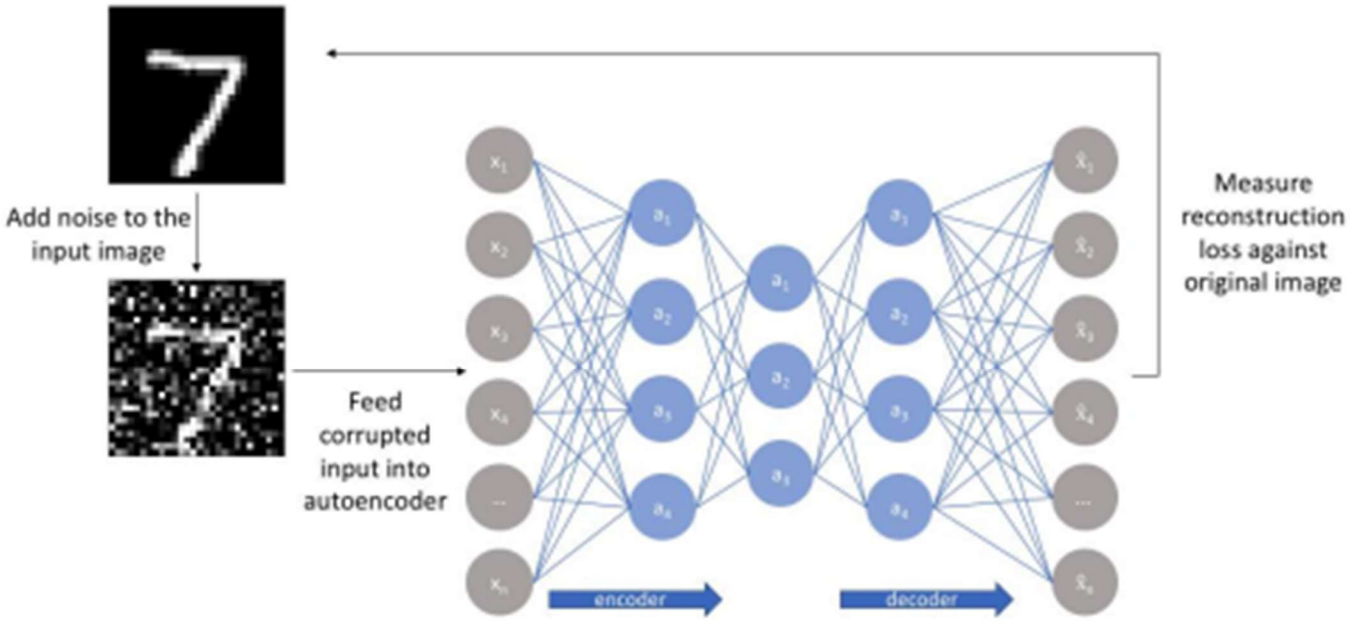


Image Credit: Abeyaantrix Edusoft

1. Encoder

It compresses the input data into a smaller, more manageable form by reducing its dimensionality while preserving important information. It has three layers which are:

- **Input Layer**: This is where the original data enters the network. It can be images, text features or any other structured data.
- **Hidden Layers**: These layers perform a series of transformations on the input data. Each hidden layer applies weights and activation functions to capture important patterns, progressively reducing the data's size and complexity.
- **Output (Latent Space)**: The encoder outputs a compressed vector known as the **latent representation** or **encoding**. This vector captures the important features of the input data in a condensed form helps in filtering out noise and redundancies.

2. Bottleneck (Latent Space)

It is the smallest layer of the network which represents the most compressed version of the input data. It serves as the information bottleneck which force the network to prioritize the most significant features. This compact representation helps the model learn the underlying structure and key patterns of the input helps in enabling better generalization and efficient data encoding.

3. Decoder

It is responsible for taking the compressed representation from the latent space and reconstructing it back into the original data form.

- **Hidden Layers**: These layers progressively expand the latent vector back into a higher-dimensional space. Through successive transformations decoder attempts to restore the original data shape and details

- **Output Layer**: The final layer produces the reconstructed output which aims to closely resemble the original input. The quality of reconstruction depends on how well the encoder-decoder pair can minimize the difference between the input and output during training.

Why use Autoencoders?

- Data compression
- Denoising
- Dimensionality reduction (like PCA, but non-linear)
- Feature extraction for downstream tasks

➢ **Implementation Steps**

Step 1: Import Libraries & Load Data

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Load data
(x_train, _), (x_test, _) = mnist.load_data()

# Normalize
x_train = x_train / 255.0
x_test = x_test / 255.0

# Flatten inputs
x_train = x_train.reshape(-1, 28*28)
x_test = x_test.reshape(-1, 28*28)
```

Step 2: Build Autoencoder

```
model = Sequential([
    Dense(64, activation='relu', input_shape=(784,)),
    Dense(32, activation='relu'),   # bottleneck (compressed)
    Dense(64, activation='relu'),
    Dense(784, activation='sigmoid')  # output same size as input
])

model.compile(optimizer='adam', loss='mse')
model.summary()
```

Step 3: Train Autoencoder

```
model.fit(x_train, x_train, epochs=5, batch_size=256, validation_split=0.1)
```

- Note: Input = Output (autoencoder learns to reconstruct).
- Loss = reconstruction error (Mean Squared Error).

Step 4: Test Reconstruction

```
decoded = model.predict(x_test[:5])

import matplotlib.pyplot as plt
for i in range(5):
    plt.subplot(2, 5, i+1)
    plt.imshow(x_test[i].reshape(28,28), cmap='gray')
    plt.title("Original")
    plt.axis("off")

    plt.subplot(2, 5, i+6)
    plt.imshow(decoded[i].reshape(28,28), cmap='gray')
    plt.title("Reconstructed")
    plt.axis("off")
plt.show()
```

➢ **Expected Results**

- **Loss curve** decreases across epochs.
- Reconstructed images should be **like** originals, though slightly blurred.
- Compressed layer (32 neurons) stores the essential features of 784-pixel input.

Experiment 4: Autoencoder (MNIST Compression)

```python
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt

# Load data
(x_train, _), (x_test, _) = mnist.load_data()
x_train, x_test = x_train/255.0, x_test/255.0
x_train = x_train.reshape(-1, 784)
x_test = x_test.reshape(-1, 784)

# Build Autoencoder
model = Sequential([
    Dense(64, activation='relu', input_shape=(784,)),
    Dense(32, activation='relu'),  # bottleneck
    Dense(64, activation='relu'),
    Dense(784, activation='sigmoid')
])

model.compile(optimizer='adam', loss='mse')
model.fit(x_train, x_train, epochs=5, batch_size=256, validation_split=0.1)

# Reconstruct
decoded = model.predict(x_test[:5])
for i in range(5):
    plt.subplot(2,5,i+1); plt.imshow(x_test[i].reshape(28,28), cmap='gray'); plt.axis("off")
    plt.subplot(2,5,i+6); plt.imshow(decoded[i].reshape(28,28), cmap='gray'); plt.axis("off")
plt.show()
```

➢ Viva Questions

1. What is an autoencoder?
2. How does an autoencoder differ from PCA?
3. What is the role of the encoder and decoder in an autoencoder?
4. Why do we use a smaller hidden layer (bottleneck)?
5. What loss function is used in autoencoders and why?
6. Can autoencoders be used for supervised tasks?
7. Explain undercomplete and overcomplete autoencoders.
8. What are denoising autoencoders?
9. Why is sigmoid often used in the output layer for images?
10. Give two real-world applications of autoencoders.